

Project Codemap: SASZombieAssaultTD

This document provides a high-level overview of the `SASZombieAssaultTD` project structure and its core components.

Top-Level Structure

- `.gitignore`: Specifies intentionally untracked files to ignore.
- `App.config`: Application configuration file.
- `SASZombieAssaultTD.csproj`: C# project file for the main application.
- `SASZombieAssaultTD.slnx`: Solution file for the project.
- `Program.cs`: Entry point of the application.
- `Engine/`: Contains the core game engine logic and systems.
- `Assets/`: Stores game assets such as configurations, HUD elements, maps, sprites, towers, UI, and zombies.
- `bgfx/`, `bimg/`, `bx/`: External rendering and utility libraries.
- `Output/`: Build output directory.
- `Project Documentation/`: Contains project-specific documentation.
- `Reports/`: Stores various reports.
- `Schemas/`: Defines data schemas.
- `SIMULATE/`: Simulation related files.
- `Snapshots/`: Project snapshots.
- `SZATD Analyzer/`: Analysis tools for the project.
- `Tests/`: Contains project tests.
- `Tools/`: Various development tools.
- `Vortice/`: External library, possibly for DirectX/graphics interop.

Engine Directory (`Engine/`)

The `Engine/` directory houses the core game logic, systems, and components.

- `EngineBootstrap.cs`: Initializes the engine.
- `EngineExceptionHandler.cs`: Handles global exceptions within the engine.
- `GameWorld.cs`: Represents the main game world.
- `GlobalUsings.cs`: Global `using` directives for the engine.

Subdirectories within `Engine/`

- `\_Archives/`: Archived or old code.
- `Achievements/`: Manages in-game achievements and challenges.
 - `AchievementCategory.cs`: Defines achievement categories.
 - `AchievementDefinition.cs`: Base definition for an achievement.
 - `AchievementInstance.cs`: Represents an instance of an achievement.
 - `ChallengeDefinition.cs`: Base definition for a challenge.
- `AI/`: Contains Artificial Intelligence related logic.
 - `AIApi.cs`: API for AI interactions.
 - `AIContext.cs`: Context for AI decision-making.
 - `AIController.cs`: Controls AI behaviors.
 - `IAIBehavior.cs`: Interface for AI behaviors.
- `Animation/`: Handles game animations.
 - `BlendTree/`, `Components/`, `Core/`, `Events/`, `Integration/`, `Systems/`: Submodules for various animation aspects.
- `Audio/`: Manages sound effects and music.
 - `AudioSystem.cs`: Core audio system.
 - `CoreAudioEngine.cs`: Low-level audio engine.
 - `PlaySound.cs`: Utility for playing sounds.
- `Camera/`: Controls in-game cameras.
 - `CameraSystem.cs`: Manages camera behavior.
- `Challenges/`: Defines and tracks in-game challenges.
 - `ChallengeDifficulty.cs`: Defines challenge difficulty levels.
- `CompilationBridge/`: Bridges between different compilation units.
 - `EnemyCompilationBridge.cs`: Specific bridge for enemy data.
- `Components/`: Defines various components used by game entities.
 - `HealthComponent.cs`: Manages entity health.
 - `MovementComponent.cs`: Handles entity movement.
 - `RenderableComponent.cs`: Marks an entity as renderable.
 - `SpriteComponent.cs`: Manages sprite rendering.

```
65     - `TransformComponent.cs`: Manages entity position, rotation, and scale.
66 - `Core/`: Fundamental utility classes and data structures.
67     - `Color.cs`, `Rect.cs`, `Size.cs`: Basic data types.
68 - `Diagnostics/`: Provides logging, debugging, and performance monitoring tools.
69     - `Debug.cs`: Debug utilities.
70     - `DiagnosticsManager.cs`: Manages diagnostic reports and monitors.
71     - `DLogger.cs`: Logging utility.
72     - `MarkdownLogWriter.cs`: Writes logs in Markdown format.
73     - `NotImplementedGuard.cs`: Helps track unimplemented features.
74 - `Dictionary/`: Contains various game definitions and enumerations.
75     - `ColorAndRenderingDef.cs`: Definitions related to color and rendering.
76     - `SpawnParametersDef.cs`: Definitions for spawning entities.
77 - `Difficulty/`: Manages game difficulty settings.
78     - `DifficultyManager.cs`: Controls overall game difficulty.
79 - `Economy/`: Handles in-game currency, resources, and transactions.
80     - `EconomyManager.cs`: Manages the game economy.
81     - `ShopSystem.cs`: Manages in-game shop functionality.
82 - `ECS/`: Entity-Component-System framework implementation.
83     - `ECSEntity.cs`: Represents a game entity.
84     - `ECSManager.cs`: Manages ECS entities and systems.
85     - `ECSSystem.cs`: Base class for ECS systems.
86     - `ECSWorld.cs`: The game world in the ECS context.
87
88 (This codemap is not exhaustive but highlights key areas. Further details can be found by
exploring individual files.)
```